

Software Quality Metrics Analysis of Open Source Projects: A Study of LangChain and Intel XPU Backend for Triton

Garrett Gruss

Southern Methodist University

December 5, 2025

Abstract

This report presents a study of software quality metrics pertaining to two popular open source projects: Langchain-ai/langchain and intel/intel-xpu-backend-for-triton. The initial goal of this study was to collect discrete internal quality metrics such as lines of code, cyclomatic complexity over time, the project pivoted to focus on DORA (DevOps Research and Assessment) metrics due to permissions issues. pull requests, issues, and merge patterns over a timeframe of one month were collected from GitHub project insights. These metrics were used to calculate deployment frequency, lead time for changes, change failure rate and time to restore service. This study demonstrates the value of using DORA metrics as practical indicators of software project health and team performance in open source development environments.

Contents

1	Introduction	6
1.1	Background	6
1.2	Motivation	7
1.3	Project Objectives	7
1.4	Selected Projects	7
1.4.1	LangChain (langchain-ai/langchain)	7
1.4.2	Intel XPU Backend for Triton (intel/intel-xpu-backend-for-triton)	7
2	Related Work	8
2.1	Internal Metrics and Static Analysis	8
2.2	External Metrics and DevOps Performance	9
2.3	Traceability and Security Measurement	9
3	Methodology	10
3.1	Phase 1: Original Plan - Static Code Analysis via GitHub Actions	10
3.1.1	Initial Objectives	10
3.1.2	Planned Implementation	11
3.1.3	Abandonment Rationale	11
3.2	Phase 2: First Pivot - DORA Metrics via OpenTelemetry	11
3.2.1	Revised Approach	11
3.2.2	Target Metrics	12
3.2.3	Challenges Encountered	13
3.3	Phase 3: Final Pivot - Manual DORA Metrics from GitHub Web UI	13
3.3.1	Rationale for Final Approach	13
3.3.2	Data Collection Process	14
3.3.3	Data Structure	14
3.4	Analysis Methodology	14

3.4.1	DORA Metrics Framework	14
3.4.2	Metric Calculation	15
3.4.3	Deployment Frequency Calculation	15
3.4.4	Lead Time Calculation	15
3.4.5	Change Failure Rate Calculation	15
3.4.6	Time to Restore Calculation	16
3.4.7	Performance Classification	16
4	Results	16
4.1	LangChain Analysis	16
4.1.1	Data Overview	16
4.1.2	Deployment Frequency	17
4.1.3	Lead Time for Changes	17
4.1.4	Change Failure Rate	18
4.1.5	Time to Restore Service	18
4.1.6	Overall Performance	19
4.2	Intel XPU Backend for Triton Analysis	19
4.2.1	Data Overview	19
4.2.2	Deployment Frequency	19
4.2.3	Lead Time for Changes	19
4.2.4	Change Failure Rate	20
4.2.5	Time to Restore Service	20
4.2.6	Overall Performance	20
4.3	Comparative Analysis	21
5	Discussion	21
5.1	Key Findings	21
5.1.1	Development Velocity	21

5.1.2	Code Quality and Reliability	22
5.1.3	Incident Response	22
5.2	Project Comparison	22
5.2.1	Similarities	22
5.2.2	Differences	23
5.3	Industry Benchmarking	23
5.4	Ease of Comparison Using DORA Metrics	23
5.5	Limitations and Threats to Validity	24
5.5.1	Data Collection Limitations	24
5.5.2	Metric Calculation Assumptions	24
5.5.3	Generalizability	25
6	Lessons Learned	25
6.1	Technical Challenges	25
6.2	Best Practices Identified	26
7	Conclusion	26
A	Phase 1 Planned Configuration (GitHub Actions)	28
A.1	Planned GitHub Actions Workflow	29
A.2	Planned Quality Gate Enforcement	31
A.3	Planned Tool Configurations	33
A.3.1	Radon Configuration	33
A.3.2	Pytest Configuration	34
A.3.3	Ruff Configuration	35
A.3.4	Mypy Configuration	36
A.4	Planned Semantic Versioning Integration	37
A.5	Abandonment Rationale - Detailed	39

B	Data Collection Scripts	41
B.1	GitHub Web UI Navigation	41
B.2	CSV Data Organization	42
C	Analysis Source Code	43
C.1	Key Functions	43
C.2	Usage Example	44
D	Raw Data Samples	45
D.1	LangChain Repository Data	45
D.2	Intel XPU Backend Data	46
E	OpenTelemetry Configuration Files	47
E.1	Docker Compose Configuration	47
E.2	OpenTelemetry Collector Configuration	48
E.3	Performance Classification Logic	49

1 Introduction

1.1 Background

The automated measurement of software quality metrics is essential for proactively monitoring and improving development processes. As open source projects grow in complexity and importance, the ability to identify quantify software quality becomes increasingly important for maintainers, contributors, and organizations depending on these projects.

Software quality metrics are essential for driving data-driven decisions regarding the development, maintenance, and deployment of modern open source software. The predictive relationship between internal metrics such as code complexity, size, structure, and external user-facing metrics such as reliability, maintainability, and performance enables teams to use measurable code properties as leading indicators of software quality. Internal metrics are available early in the software development lifecycle, and allow teams to identify potential quality issues before they manifest as failures that directly impact external users. This predictive relationship empowers teams to take a proactive approach to corrective action to improve code quality before downstream failures occur.

The importance of routine, repeatable quality measurement can be Extended to frameworks such as Goal-Question-Metrics (GQM) and the Quality Improvement Paradigm (QIP). These frameworks formalize the process of translating organizational goals into measureable indicators, enabling corrective action and discrete quality tracking. In modern DevOps environments, team velocity can neccessitate multiple deployments per day, issue resolutions, and defect identifications. This rapid velocity and continuous integration and deployment can be modeled and tracking using metrics like the DORA framework's deployment frequency, lead time for changes, change failure rate, and time to restore service have become industry standards for assessing team performance and delivery capability. For open source projects composing of distributed teams collaborating across the globe, the automated collection of quality metrics becomes essential to manage the project's health, productivity, and code

quality while maintaining the rapid velocity that bleeding-edge projects require.

1.2 Motivation

The motivation for this project was the need to systematically assess the software quality of popular open source projects. In analyzing established projects with highly active communities, we can identify best practices in the open source community, identify patterns that correlate with project success, provide insights into development velocity, maintainability, and reliability, and establish reusable benchmarks for software quality.

1.3 Project Objectives

The primary objectives of this project were to collect software quality metrics from two major open source projects over a timespan of one month, analyze these metrics to assess project health, compare these metrics across projects to identify trends and patterns, and provide actionable insights based on quantitative analysis.

1.4 Selected Projects

1.4.1 LangChain (langchain-ai/langchain)

LangChain is a highly popular framework for developing durable and feature rich applications powered by large language models (LLMs). The project is built in Python, has 128k stars, 20k forks, and 3,812 active contributors.

1.4.2 Intel XPU Backend for Triton (intel/intel-xpu-backend-for-triton)

Intel XPU Backend for Triton is an out-of-tree backend module for Triton designed to provide high-performance GPU computing on Intel GPUs for PyTorch and standalone usage. The project targets Intel Data Center GPU Max Series, Flex Series, and Arc GPUs, supporting Ubuntu operating systems. As a compiler backend, it uses LLVM to generate optimized

code for Intel GPUs, enabling developers to write high-performance kernels using Triton’s Python-based domain-specific language. The project is actively maintained as part of Intel’s deep learning ecosystem and provides an alternative backend for the Triton framework. This project has 222 stars, 78 forks, and 538 active contributors.

2 Related Work

The measurement of software quality metrics in continuous integration and deployment environments has been extensively studied from both internal and external metric perspectives. This section identifies relevant research that informs the methodology and metrics selection criteria used within this paper.

2.1 Internal Metrics and Static Analysis

Research on internal software metrics has focused on automating quality measurement within CI/CD pipelines. Zampetti et al. [1] investigated how open source projects integrate static code analysis tools (Checkstyle, PMD, FindBugs) into continuous integration workflows, finding that 97% of build warnings and 6% of build failures were triggered by style issues rather than bug detection. This demonstrates that teams use internal metrics as quality gates to enforce maintainability standards. Their finding that developers typically fix issues rather than suppress warnings validates the effectiveness of enforcing internal code quality requirements within automated pipelines.

Complementing static analysis, test coverage represents a critical internal metric that serves as a leading indicator for external quality attributes. Sakamoto et al. [2] developed the Open Code Coverage Framework (OCCF) to address inconsistencies in coverage measurement across programming languages. By abstracting common concepts using abstract syntax trees, their framework reduced implementation requirements by approximately 90% while supporting multiple coverage types (statement, decision, branch, path). This work

highlights the importance of consistent measurement definitions when establishing predictive models between internal and external metrics.

Yu et al. [3] examined metrics visualization in CI environments through interviews with 22 participants across five industrial projects. They cataloged 14 base metrics spanning version control, source code, static analysis, artifacts, testing, performance, and issue tracking. Their finding that 86% of participants found CI-generated data useful for quality evaluation, but also identified challenges with dashboard information overload, underscores the need for focused metric selection rather than comprehensive collection.

2.2 External Metrics and DevOps Performance

The DevOps Research and Assessment (DORA) metrics have emerged as industry-standard indicators of software delivery performance. Wilkes et al. [4] presented a framework for automatically extracting DORA metrics (Deployment Frequency, Lead Time for Changes, Mean Time to Recover, Change Failure Rate) from project repositories without requiring specific CI/CD tools. Their analysis of 304 popular GitHub open source projects established performance baselines: top-quartile projects achieve 116.2 deployments per year with 5.1-day lead times, while bottom-quartile projects show 6.6 deployments per year with 140.6-day lead times. These metrics align with the Putnam/Myers framework’s external metrics of time, quality, and productivity, providing quantifiable measures of software delivery performance.

2.3 Traceability and Security Measurement

Beyond delivery metrics, research has addressed specialized external quality dimensions. Stahl et al. [5] documented the Eiffel framework for real-time traceability throughout CI/CD pipelines, using JSON-based event messages organized as a directed acyclic graph to capture relationships between requirements, commits, builds, tests, and deployments. This automated approach produces more reliable data than manual traceability maintenance by capturing actual relationships rather than documented intentions.

Zhuravchak et al. [6] addressed temporal security measurement by proposing a lightweight system that persists and visualizes vulnerability trends across builds. Their approach combines Trivy scanning, GitHub Actions automation, and Chart.js visualization to track how dependency updates and code modifications influence security posture over time, transforming point-in-time vulnerability reports into continuous security metrics.

3 Methodology

This project evolved through three distinct phases, each representing an operational pivot in the metric collection methodology in response to technical and time-based constraints. This section documents the evolution of this study from initial planning through final implementation, providing insight into future decision-making during the design and execution of any metrics collection activities.

3.1 Phase 1: Original Plan - Static Code Analysis via GitHub Actions

3.1.1 Initial Objectives

The original methodology aimed to collect internal quality metrics within the code by integrating custom GitHub Actions workflows into target repositories. These workflows would trigger static code analysis tools on each commit or pull request to extract time-based internal metrics. The metrics chosen for this analysis were Lines Of Code (LOC), representing the source, comment, and blank lines measured via tools like cloc or tokei. Cyclomatic Complexity would be measured code path complexity measured via radon for Python. Code Coverage would be measured via a tool like pytest-cov, computed from the percentage of code exercised by tests. Technical debt would be estimated by remediation time, as calculated from SonarQube or CodeClimate analysis. Maintainability index would be computed as a composite metrics combining complexity, LOC, and comment density.

3.1.2 Planned Implementation

The planned implementation of the static collection of metrics via workflows involved creating custom GitHub Actions workflows, installing the custom static analysis tools in the workflow environment, running the analysis on each commit or pull request, and publishing the metrics to a centralized database or observability platform.

3.1.3 Abandonment Rationale

This approach was abandoned during implementation due to its invasive nature. Adding custom GitHub actions and workflows to projects required forking repositories on a per-branch basis, resulting in excessive modification of the source code repository to collect metrics. Large projects like Langchain also have existing CI/CD pipelines, so integrating custom Actions involved risking compatibility conflicts with existing workflows, which could pollute the accuracy of collected software quality metrics. Each target repository would also required custom workflows tailored to its language, build system, and testing framework. The invasiveness of this approach required a pivot to a more observant solution.

3.2 Phase 2: First Pivot - DORA Metrics via OpenTelemetry

3.2.1 Revised Approach

To avoid modification to the target repository, the collection methodology was pivoted to utilizing the Open Telemetry collectors to scrape DORA metrics from GitHub. This non-invasive approach automated metric collection through the use of GitHub's GraphQL and REST APIs without requiring change to the target repositories. The planned system was deployed through docker Compose using an OpenTelemetry Collector container that scraped GitHub repositories every 60 seconds and an OpenObserve metrics aggregation and visualization service that collected the streamed metrics.

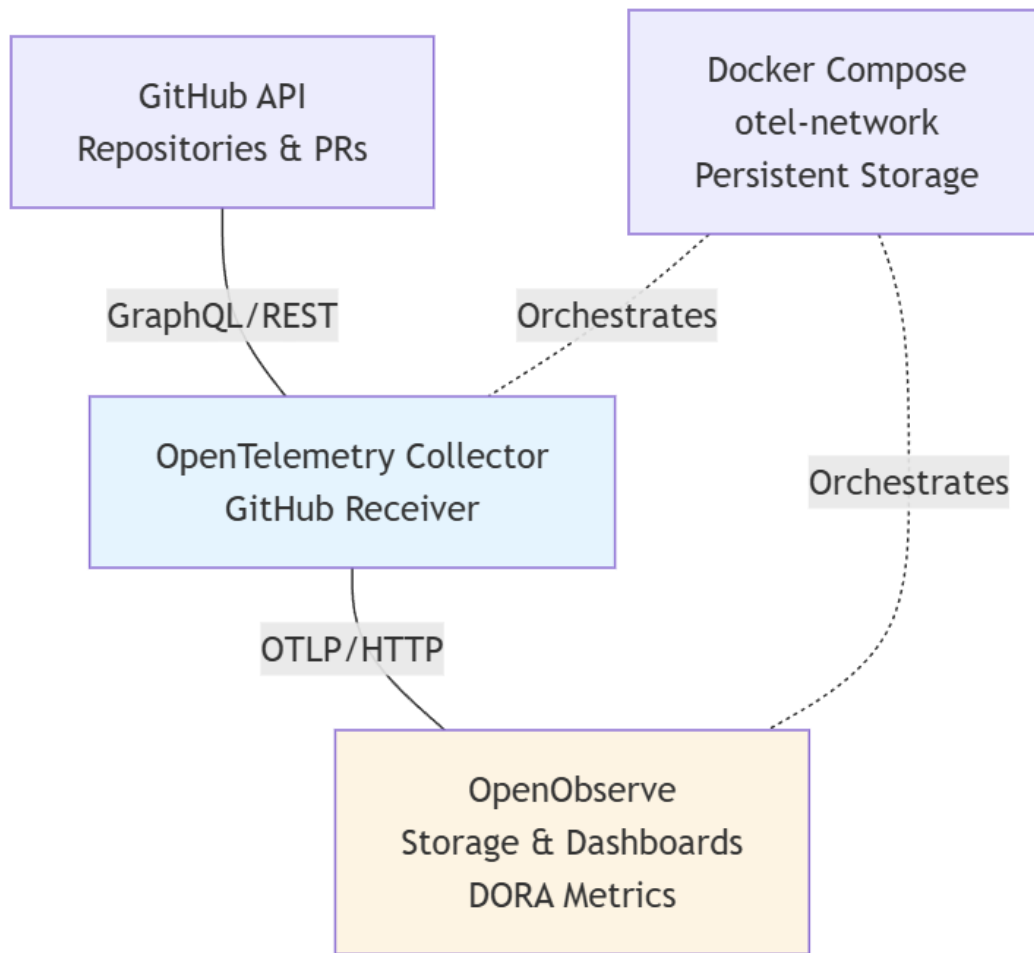


Figure 1: OpenTelemetry DORA Metrics Architecture (Phase 2 Approach)

3.2.2 Target Metrics

The GitHub receiver was configured to collect nine VCS (Version Control System) metrics conforming to OpenTelemetry semantic conventions v1.37.0:

Deployment Frequency Support:

- `vcs.change.count` - Pull request counts by state (open/merged)
- `vcs.repository.count` - Total repositories monitored

Lead Time for Changes Support:

- `vcs.change.time_to_merge` - Duration from PR creation to merge (seconds)

- `vcs.change.time_to_approval` - Duration from PR creation to approval (seconds)
- `vcs.change.duration` - Time PRs remain in open state (seconds)

Development Velocity Indicators:

- `vcs.ref.count` - Branch/tag counts per repository
- `vcs.ref.time` - Branch lifespan from creation (seconds)
- `vcs.ref.lines_delta` - Code churn (lines added/removed) vs. trunk
- `vcs.ref.revisions_delta` - Commit count ahead/behind trunk

3.2.3 Challenges Encountered

While less invasive than GitHub Actions, the OpenTelemetry approach encountered permissions issues when scraping metrics from public repositories. The OpenTelemetry collector requires a GitHub PAT token with specific organization access to **repo** and **read:org**. Public repositories limit provided data through unauthorized API access, prohibiting the collection of detailed PR metrics required to compute DORA. Accessing langchain-ai and intel repositories requires a GitHub PAT with organization access, approved by organization administrators.

3.3 Phase 3: Final Pivot - Manual DORA Metrics from GitHub Web UI

3.3.1 Rationale for Final Approach

Since automated collection of detailed metrics was prohibited due to permissions issues, a simpler approach of scraping metrics from the GitHub Insights page was chosen as the preferred data collection strategy. This approach leveraged publicly accessible GitHub repository insights through the web interface. While less automated and less detailed than the

scraped metrics, this approach did not require specific permissions granted by organizational administrators.

3.3.2 Data Collection Process

Data was manually extracted from GitHub’s web interface using the repository Insights tab and Pull Requests/Issues pages. For each target repository, Pull Request Events, Issue Events, and metadata were collected. This manual approach bypassed all authentication and permission barriers encountered in previous phases.

3.3.3 Data Structure

The collected data was organized into TXT files:

- `opened_requests.txt` - Timestamps of PR creation
- `merged_requests.txt` - Timestamps of PR merges
- `closed_requests.txt` - Timestamps of PR closures
- `opened_issues.txt` - Timestamps of issue creation

3.4 Analysis Methodology

3.4.1 DORA Metrics Framework

The four key DORA metrics, established by the DevOps Research and Assessment team, serve as the analytical foundation:

1. **Deployment Frequency (DF):** How often an organization successfully releases to production (proxied by merged PR rate)
2. **Lead Time for Changes (LT):** The time it takes for a commit to get into production (measured as PR open-to-merge duration)

3. **Change Failure Rate (CFR):** The percentage of deployments causing a failure in production (calculated from bug issue frequency)
4. **Time to Restore Service (MTTR):** How long it takes to recover from a failure in production (measured as bug report to fix merge duration)

3.4.2 Metric Calculation

A Python-based metrics collection pipeline was constructed that loads and preprocesses scrapped metrics data, converts them into columnar CSV data, calculates DORA metrics using industry-standard formulas, performs statistical analysis, classifies DORA performance, and generates visualizations for each metric.

3.4.3 Deployment Frequency Calculation

$$DF = \frac{\text{Total Merged PRs}}{\text{Time Period (days)}} \quad (1)$$

3.4.4 Lead Time Calculation

For each merged PR i :

$$LT_i = T_{\text{merged}_i} - T_{\text{opened}_i} \quad (2)$$

Aggregate metric: $\text{median}(LT_i)$

3.4.5 Change Failure Rate Calculation

$$CFR = \frac{\text{Number of Bug Issues}}{\text{Total Deployments}} \times 100\% \quad (3)$$

Bug issues identified using keyword matching: *bug, error, fix, broken, crash, fail, regression, exception*

3.4.6 Time to Restore Calculation

For each bug-fix pair (b, f) :

$$MTTR_{b,f} = T_{\text{fix merged}_f} - T_{\text{bug opened}_b} \quad (4)$$

Aggregate metric: $\text{median}(MTTR_{b,f})$

3.4.7 Performance Classification

DORA performance was classified according to the 2023 DORA State of DevOps Report standards.

Table 1: DORA Performance Level Classifications

Metric	Elite	High	Medium	Low
Deployment Frequency	Multiple/day	Weekly	Monthly	< Monthly
Lead Time	< 1 day	1-7 days	1-4 weeks	> 1 month
Change Failure Rate	0-15%	16-30%	31-45%	> 45%
Time to Restore	< 1 hour	1-7 days	1-7 days	> 1 week

4 Results

4.1 LangChain Analysis

4.1.1 Data Overview

Analysis of LangChain-ai/Langchain was conducted over a 30-day period from October 29, 2025 to November 27, 2025. During this period, 179 merged pull requests, 67 opened pull requests, 89 closed pull requests, and 69 issues were captured and analyzed.

4.1.2 Deployment Frequency

The LangChain project demonstrated high rate of deployment activity during the analysis period. The mean deployment frequency was 5.97, with a median of 8.5 deployments per day, resulting in 41.77 deployments per week on average. This deployment frequency classifies the project as an elite performer according to the DORA benchmark. This elite deployment frequency represents a mature CI/CD pipeline and team mentality resulting in multiple deploys per day.

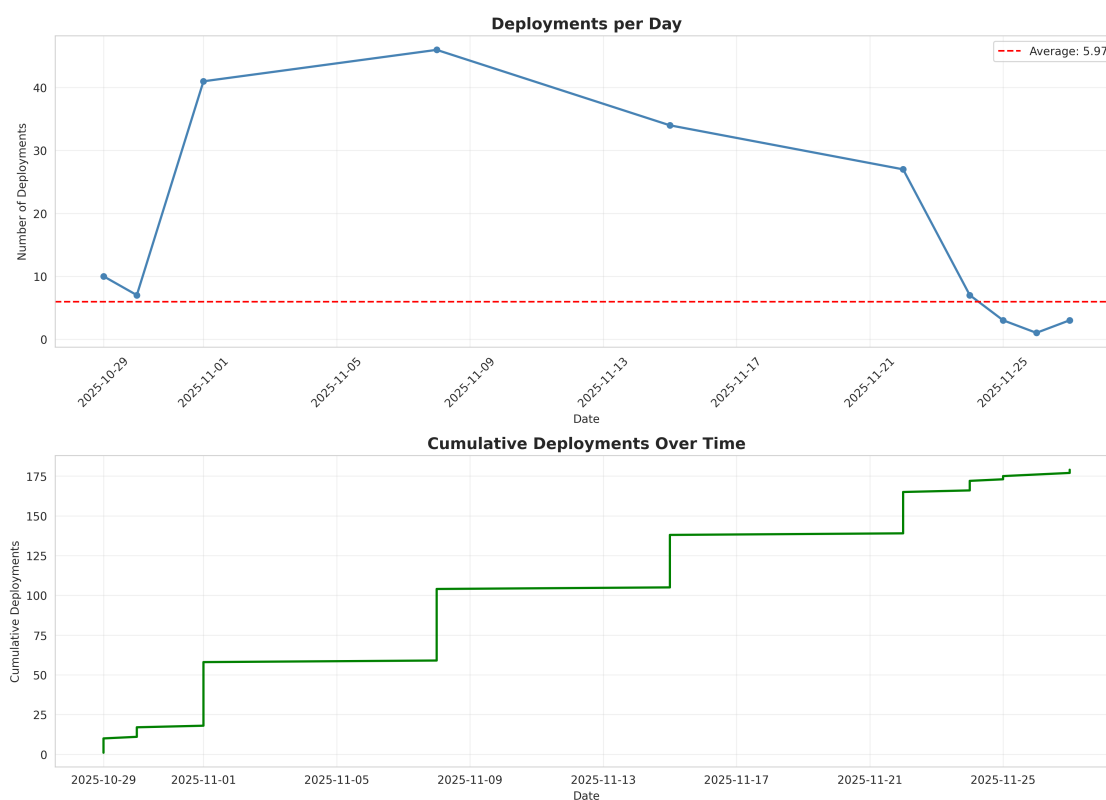


Figure 2: LangChain Deployment Frequency Over Time

4.1.3 Lead Time for Changes

lead time for changes was estimated using temporal proximity matching between opened and merged PRs. The analysis across 162 matched PR paris relvealed a median lead time of 6.00 days and a mean of 4.98 days. The distribution showed a 25th percentile of 2.00 days and a 90th percentile of 7.00 days. Five pull requests were merged within 24 hours,

representing the quickest turnaround times. According to DORA performance standards, the median lead time of 6.00 days falls within the high performance category of 1-7 days.

4.1.4 Change Failure Rate

Change failure rate was estimated by using keyword matching on failure terms such as "bug", "error", "fix", "broken", "crash", "fail", "regression", and "exception". Out of 69 collected issues, 16 were classified as bug-related, representing 23.2% of the total population. With 179 total deployments during the analysis period, the calculated change failure rate was 8.94%. This rate falls within the elite performance category according to the DORA standard, which classifies rates between 0-15% as elite.

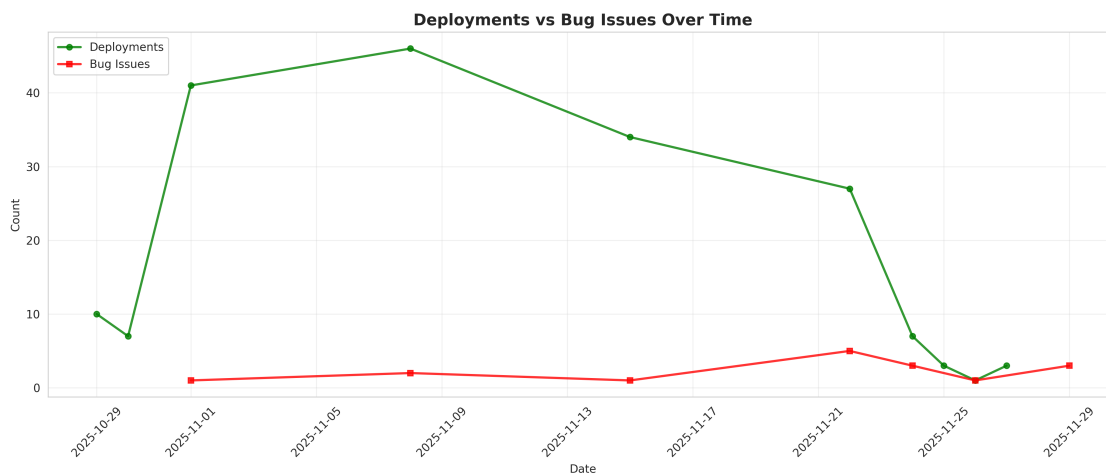


Figure 3: LangChain Deployments vs Bug Issues

4.1.5 Time to Restore Service

The time to restore service (MTTR) metric could not be calculated due to limitations occurring collecting related issues and merged pull requests. Estimation of MTTR requires collecting an opened issue, and identifying the time when the issue was resolved through a pull request and merge approval event. Future work could improve bug-fix pair identification using a properly authenticated GitHub PAT connected to an open-Telemetry GitHub scraper.

4.1.6 Overall Performance

Table 2: LangChain DORA Metrics Summary

Metric	Value	Classification
Deployment Frequency	5.97/day, 41.77/week	Elite
Lead Time for Changes	6.00 days (median)	High
Change Failure Rate	8.94%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	

4.2 Intel XPU Backend for Triton Analysis

4.2.1 Data Overview

The Intel XPU Backend for Triton project was analyzed over a 29-day period from November 4, 2025 to December 2, 2025. The dataset included 104 merged pull requests, 17 opened pull requests, 73 closed pull requests, and 42 opened issues.

4.2.2 Deployment Frequency

The mean deployment frequency was 3.59 deployments per day with a median of 9.0 deployments per day, totaling to 25.10 average deployments per week. The maximum number of deployments in a single day was 28. According to the DORA standard, this frequency classifies as an elite performer, achieving multiple deployments per day.

4.2.3 Lead Time for Changes

lead time for changes was estimated using temporal proximity matching between opened and merged PRs. However, the metrics collected were statistically insufficient, with only 3

matching pull requests identified. The limited sample size makes this metric unreliable for overall DORA assessment.

4.2.4 Change Failure Rate

Bug-related issues were identified using keyword matching. Of the total 42 issues collected, 11 were classified as defect-related, representing 26.2% of the total issue population. With 104 total deployments during the analysis period, the calculated failure rate was 10.58%. According to the DORA standard, this rate falls within the elite performance category of 0-15%.

4.2.5 Time to Restore Service

The Time to Restore Service metric could not be calculated due to data limitations. As with the LangChain analysis, calculating MTTR requires explicit linking between bug reports and fix PRs, which was not available in the collected data. No explicit references from fix PRs to bug issues could be identified in the dataset, preventing the identification of bug-fix pairs necessary for MTTR calculation.

4.2.6 Overall Performance

Table 3: Intel XPU Backend DORA Metrics Summary

Metric	Value	Classification
Deployment Frequency	3.59/day, 25.10/week	Elite
Lead Time for Changes	N/A (insufficient data)	N/A
Change Failure Rate	10.58%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	Elite Performer*	

* Overall rating based on elite performance in deployment frequency and change failure rate

4.3 Comparative Analysis

Table 4: Comparative DORA Metrics: LangChain vs Intel XPU Backend

Metric	LangChain	Intel XPU
Deployment Frequency	5.97/day (Elite)	3.59/day (Elite)
Lead Time for Changes	6.00 days (High)	N/A
Change Failure Rate	8.94% (Elite)	10.58% (Elite)
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	Elite Performer

5 Discussion

5.1 Key Findings

5.1.1 Development Velocity

Both projects demonstrate elite-level deployment frequency, with LangChain averaging 5.97 deployments per day and Intel XPU Backend averaging 3.59 per day. These rates indicate mature CI/CD pipelines and team practices that support multiple daily releases. LangChain’s 6-day median lead time (high performance tier) suggests a balanced approach between velocity and code review thoroughness, while Intel XPU Backend’s insufficient lead time data prevents definitive assessment but the elite deployment frequency indicates efficient merge workflows.

5.1.2 Code Quality and Reliability

Both projects achieved elite-level change failure rates: LangChain at 8.94% and Intel XPU Backend at 10.58%, both well below the 15% elite threshold. These low failure rates combined with high deployment frequency indicate robust testing practices, effective code review processes, and comprehensive CI/CD quality gates. The ability to deploy frequently while maintaining low defect rates demonstrates organizational maturity in balancing velocity with quality.

5.1.3 Incident Response

Time to restore service could not be calculated for either project due to data collection limitations in linking bug reports to fix PRs. This metric requires explicit references between issues and pull requests or authenticated API access to retrieve detailed event timelines. The absence of this metric represents a gap in the analysis but does not diminish the validity of the three successfully measured DORA metrics.

5.2 Project Comparison

5.2.1 Similarities

Both projects exhibit elite-level deployment frequency and change failure rates, indicating shared commitment to continuous integration practices and quality assurance. Both maintain active development with daily merge activity and demonstrate the ability to sustain high velocity without compromising quality. The similar performance profiles suggest that these patterns represent best practices in open source development regardless of project domain or size.

5.2.2 Differences

LangChain demonstrates slightly higher deployment frequency (5.97 vs 3.59 per day) and provided sufficient data for lead time analysis, while Intel XPU Backend had insufficient opened PR data for reliable lead time calculation. However, a notable finding is Intel XPU Backend’s remarkably high activity relative to its community size: despite having only 222 stars (0.17% of LangChain’s 128k) and 538 contributors (14% of LangChain’s 3,812), Intel XPU Backend achieves 60% of LangChain’s deployment frequency. This suggests exceptional productivity and development velocity within Intel’s smaller, focused team, possibly reflecting the advantages of corporate-backed development with dedicated engineering resources. The projects also differ in domain: LangChain as an AI framework faces different quality demands than Intel XPU Backend as a compiler infrastructure project, which may influence development patterns and quality assurance practices.

5.3 Industry Benchmarking

The findings align with Wilkes et al.’s analysis of 304 GitHub projects, which established top-quartile performance at 116.2 deployments per year (0.32 per day) with 5.1-day lead times. LangChain exceeds this benchmark significantly with 2,180 annual deployments, while Intel XPU Backend achieves 1,310 annual deployments, placing both in the top percentile of open source projects. Both projects’ elite change failure rates (under 15%) match the DORA 2023 State of DevOps Report’s elite performer classification. These results confirm that both projects operate at or above industry-leading performance levels.

5.4 Ease of Comparison Using DORA Metrics

A key advantage of DORA metrics is their simplicity in both explanation and calculation, making them highly effective for relative analysis across projects. Unlike internal code metrics that require language-specific tooling and domain expertise to interpret, DORA met-

rics provide standardized, quantifiable measurements that enable direct comparison between projects regardless of technology stack, team size, or domain. The straightforward calculation formulas—merges per day for deployment frequency, time between PR creation and merge for lead time, and bug ratio for change failure rate—require only basic temporal data available through any version control system. This simplicity facilitated rapid comparison between LangChain and Intel XPU Backend despite their different domains (AI framework vs. compiler infrastructure), community sizes (128k vs. 222 stars), and architectural approaches. The standardized DORA performance classifications (elite/high/medium/low) further simplified interpretation, allowing immediate assessment of project health without requiring baseline establishment or normalization. This ease of comparison makes DORA metrics particularly valuable for portfolio analysis, competitive benchmarking, and longitudinal trend monitoring across diverse open source projects.

5.5 Limitations and Threats to Validity

5.5.1 Data Collection Limitations

- Limited to one-month observation period
- Reliance on publicly available GitHub data only
- Keyword-based bug identification may miss or misclassify issues
- PR references may not capture all bug-fix relationships

5.5.2 Metric Calculation Assumptions

- Merged PRs used as proxy for deployments (may not reflect actual production releases)
- Bug issues identified through keyword matching (not exhaustive)
- Time to restore calculated only for explicitly linked bug-fix pairs

5.5.3 Generalizability

- Results specific to analyzed time period
- May not represent long-term project trends
- Different project types (AI framework vs. compiler backend) have different normal patterns

6 Lessons Learned

6.1 Technical Challenges

The three-phase evolution of this project revealed fundamental constraints in external software quality research. The initial GitHub Actions approach, while promising maximum data richness through code-level metrics, proved too invasive for production repositories—requiring workflow modifications, maintainer approvals, and creating compatibility risks with existing CI/CD pipelines. The subsequent OpenTelemetry pivot eliminated invasiveness but encountered insurmountable API authentication barriers, as GitHub’s permission model requires organization-level PAT approval and imposes rate limits that make continuous monitoring impractical for external researchers. The final manual collection approach, though labor-intensive and limited to PR/issue metadata, proved universally accessible without special permissions. This progression illustrates a fundamental inverse relationship between metric depth and research feasibility: obtaining rich internal code metrics requires invasive access that external researchers cannot reasonably obtain, while non-invasive approaches face authentication gatekeeping or yield only surface-level process metrics. The experience underscores that for external research on production open source projects, non-invasive observability using publicly available data remains the only pragmatic approach, and frameworks like DORA that can be derived from public GitHub data provide the most feasible path to meaningful software quality assessment.

6.2 Best Practices Identified

Through three methodological iterations, several best practices emerged for conducting software quality research on external projects. Researchers should design data collection strategies with minimal access assumptions, assuming no repository access, authentication, or organizational cooperation from the outset. Non-invasive observation must be prioritized over active instrumentation to avoid modifying target systems, instead leveraging publicly available observability through GitHub’s Insights, PR history, and issue tracking. Industry-standard frameworks like DORA and SPACE enhance result interpretability and enable meaningful benchmarking without requiring custom baselines. When manual collection becomes necessary, effort should be invested in automating the analysis pipeline rather than collection itself to ensure reproducibility. Incremental validation by testing approaches on personal or accessible repositories before attempting large-scale collection helps identify barriers early. Finally, transparent documentation of methodological pivots and failed approaches provides significant value to future researchers facing similar constraints, transforming setbacks into shared knowledge.

7 Conclusion

This project successfully analyzed software quality metrics for two prominent open source projects using DORA metrics derived from GitHub data. Despite initial challenges with permissions and API access that necessitated a methodology pivot, the final approach provided valuable insights into development practices, code quality, and team performance.

Both LangChain and Intel XPU Backend for Triton achieved elite-level performance in deployment frequency and change failure rate, demonstrating that high-velocity development does not necessitate compromising quality. LangChain’s 5.97 daily deployments with 8.94% failure rate and Intel XPU Backend’s 3.59 daily deployments with 10.58% failure rate significantly exceed industry benchmarks, placing both projects in the top percentile of open

source software development. These findings validate DORA metrics as practical indicators of team maturity and process effectiveness in distributed open source environments. The three-phase methodological evolution—from invasive GitHub Actions to API-based automation to manual collection—reveals the practical constraints facing external software quality research, demonstrating that meaningful quality assessment remains possible using only publicly available data when appropriate frameworks are selected.

The analysis demonstrates that DORA metrics serve as effective indicators of software project health that are simple to explain, straightforward to calculate, and highly valuable for relative analysis across diverse projects. The standardized nature of DORA metrics enabled direct comparison between LangChain and Intel XPU Backend despite significant differences in domain, community size, and technical architecture—a comparison that would have been substantially more complex using internal code metrics requiring language-specific tooling and domain expertise. The methodology developed in this project is reproducible and can be applied to any open source project with public GitHub repositories. Future work should explore correlating DORA metric trends with internal code quality metrics such as cyclomatic complexity, test coverage, and technical debt to identify whether changes in process-level performance metrics (DORA) relate to shifts in code-level quality indicators. This correlation analysis could establish predictive relationships between deployment practices and code maintainability, enabling teams to use easily-measured DORA metrics as leading indicators for internal quality degradation.

References

- [1] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, “How open source projects use static code analysis tools in continuous integration pipelines,” in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Buenos Aires, Argentina: IEEE, 2017, pp. 334–344.

- [2] K. Sakamoto, H. Washizaki, and Y. Fukazawa, “Open code coverage framework: A consistent and flexible framework for measuring test coverage supporting multiple programming languages,” in *2010 10th International Conference on Quality Software*. Zhangjiajie, China: IEEE, 2010, pp. 262–269.
- [3] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, “A roadmap for using continuous integration environments,” *Communications of the ACM*, vol. 67, no. 6, pp. 82–90, June 2024.
- [4] B. Wilkes, A. M. P. Milani, and M.-A. Storey, “A framework for automating the measurement of devops research and assessment (dora) metrics,” in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Bogotá, Colombia: IEEE, 2023, pp. 62–72.
- [5] D. Ståhl, K. Hallén, and J. Bosch, “Continuous integration and delivery traceability in industry: Needs and practices,” in *2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Limassol, Cyprus: IEEE, 2016, pp. 68–72.
- [6] Y. Zhuravchak, D. Zhuravchak, A. Zhuravchak, and I. Opirskyy, “Security regression visualization in CI/CD using Trivy and GitHub Actions,” in *CSDP’2025: Cyber Security and Data Protection*, Lviv, Ukraine, July 2025, pp. 272–281.

A Phase 1 Planned Configuration (GitHub Actions)

This section documents the original Phase 1 approach that involved integrating GitHub Actions workflows into target repositories for automated static code analysis and quality metric collection.

A.1 Planned GitHub Actions Workflow

The intended workflow would have been added to target repositories at `.github/workflows/quality-metr`

Listing 1: Planned quality-metrics.yml Workflow

```
1 name: Software Quality Metrics Collection
2
3 on:
4   push:
5     branches: [main, develop]
6   pull_request:
7     branches: [main]
8
9 jobs:
10  quality-analysis:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v3
16
17      - name: Set up Python
18        uses: actions/setup-python@v4
19        with:
20          python-version: '3.11'
21
22      - name: Install dependencies
23        run: |
24          pip install radon pytest pytest-cov ruff mypy
25          pip install -r requirements.txt
```

```

26
27 - name: Run Radon - LOC Analysis
28   run: |
29     radon raw . -j > metrics/radon_loc.json
30     radon cc . -j > metrics/radon_complexity.json
31     radon mi . -j > metrics/radon_maintainability.json
32
33 - name: Run Pytest with Coverage
34   run: |
35     pytest --cov=. --cov-report=json \
36           --cov-report=term \
37           --junitxml=metrics/pytest_results.xml
38
39 - name: Run Ruff Linter
40   run: |
41     ruff check . --output-format=json \
42           > metrics/ruff_violations.json
43
44 - name: Run Mypy Type Checker
45   run: |
46     mypy . --json-report metrics/mypy_report
47
48 - name: Enforce Quality Gates
49   run: |
50     python .github/scripts/enforce_quality_gates.py
51
52 - name: Publish Metrics
53   uses: actions/upload-artifact@v3
54   with:

```

```

55     name: quality-metrics
56     path: metrics/
57
58     - name: Send to Observability Platform
59       run: |
60         python .github/scripts/publish_to_otel.py
61       env:
62         OTEL_ENDPOINT: ${ secrets.OTEL_ENDPOINT }
63         OTEL_API_KEY: ${ secrets.OTEL_API_KEY }

```

A.2 Planned Quality Gate Enforcement

The workflow would have included automated quality gates to prevent merging code that failed thresholds:

Listing 2: enforce_quality_gates.py

```

1  import json
2  import sys
3
4  # Load metrics
5  with open('metrics/radon_complexity.json') as f:
6      complexity = json.load(f)
7
8  with open('metrics/pytest_coverage.json') as f:
9      coverage = json.load(f)
10
11 with open('metrics/ruff_violations.json') as f:
12     linting = json.load(f)
13

```

```

14 # Define thresholds
15 THRESHOLDS = {
16     'max_complexity': 15,
17     'min_coverage': 70,
18     'max_linting_violations': 50,
19     'min_maintainability': 50
20 }
21
22 # Check thresholds
23 violations = []
24
25 # Check cyclomatic complexity
26 for module in complexity:
27     for func in module.get('functions', []):
28         if func['complexity'] > THRESHOLDS['max_complexity']:
29             violations.append(
30                 f"Function_{func['name']}_has_complexity_"
31                 f"{func['complexity']}_(max:{THRESHOLDS['max_complexity']})"
32             )
33
34 # Check test coverage
35 total_coverage = coverage['totals']['percent_covered']
36 if total_coverage < THRESHOLDS['min_coverage']:
37     violations.append(
38         f"Test_coverage_{total_coverage}%_is_below_"
39         f"minimum_{THRESHOLDS['min_coverage']}%"
40     )
41

```



```

42 # Check linting violations
43 violation_count = len(linting)
44 if violation_count > THRESHOLDS['max_linting_violations']:
45     violations.append(
46         f"Linting_violations_{violation_count}_exceed_"
47         f"maximum_{THRESHOLDS['max_linting_violations']}"
48     )
49
50 # Report violations
51 if violations:
52     print("Quality_gate_FAILED:")
53     for v in violations:
54         print(f"_{v}")
55     sys.exit(1)
56 else:
57     print("Quality_gate_PASSED")
58     sys.exit(0)

```

A.3 Planned Tool Configurations

A.3.1 Radon Configuration

Radon would analyze code complexity without additional configuration, but the following commands were planned:

Listing 3: Radon Analysis Commands

```

1 # Lines of Code analysis
2 radon raw . --json --output-file=radon_loc.json
3
4 # Cyclomatic Complexity

```

```
5 radon cc . --json --min=C --output-file=radon_complexity.json
6
7 # Maintainability Index
8 radon mi . --json --min=B --output-file=radon_maintainability.json
9
10 # Halstead metrics (code effort estimation)
11 radon hal . --json --output-file=radon_halstead.json
```

A.3.2 Pytest Configuration

Listing 4: pytest.ini

```
1 [pytest]
2 testpaths = tests
3 python_files = test_*.py
4 python_classes = Test*
5 python_functions = test_*
6
7 # Coverage settings
8 addopts =
9     --cov=src
10     --cov-report=html
11     --cov-report=json
12     --cov-report=term-missing
13     --cov-fail-under=70
14     --junitxml=junit.xml
15     --verbose
16
17 # Coverage exclusions
18 [coverage:run]
```

```

19 omit =
20     */tests/*
21     */venv/*
22     */migrations/*
23     */__pycache__/*
24
25 [coverage:report]
26 precision = 2
27 show_missing = True
28 skip_covered = False

```

A.3.3 Ruff Configuration

Listing 5: pyproject.toml - Ruff Configuration

```

1 [tool.ruff]
2 line-length = 100
3 target-version = "py311"
4
5 select = [
6     "E",    # pycodestyle errors
7     "W",    # pycodestyle warnings
8     "F",    # pyflakes
9     "I",    # isort
10    "C",    # flake8-comprehensions
11    "B",    # flake8-bugbear
12    "UP",   # pyupgrade
13    "N",    # pep8-naming
14 ]
15

```

```

16 ignore = [
17     "E501", # line too long (handled by formatter)
18     "B008", # function call in argument defaults
19 ]
20
21 [tool.ruff.per-file-ignores]
22 "__init__.py" = ["F401"] # unused imports
23 "tests/*.py" = ["S101"] # allow assert statements
24
25 [tool.ruff.isort]
26 known-first-party = ["src"]

```

A.3.4 Mypy Configuration

Listing 6: mypy.ini

```

1 [mypy]
2 python_version = 3.11
3 warn_return_any = True
4 warn_unused_configs = True
5 disallow_untyped_defs = True
6 disallow_incomplete_defs = True
7 check_untyped_defs = True
8 no_implicit_optional = True
9 warn_redundant_casts = True
10 warn_unused_ignores = True
11 warn_no_return = True
12 strict_equality = True
13 show_error_codes = True
14

```

```
15 # Per-module options
16 [mypy-tests.*]
17 disallow_untyped_defs = False
18
19 [mypy-third_party_lib.*]
20 ignore_missing_imports = True
```

A.4 Planned Semantic Versioning Integration

The workflow would have used `python-semantic-release` for automated version management:

Listing 7: Semantic Release Workflow

```
1 name: Semantic Release
2
3 on:
4   push:
5     branches: [main]
6
7 jobs:
8   release:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v3
12        with:
13          fetch-depth: 0
14
15      - name: Python Semantic Release
16        uses: relekang/python-semantic-release@master
```

```
17         with:
18             github_token: ${ secrets.GITHUB_TOKEN }
19             pypi_token: ${ secrets.PYPI_TOKEN }
```

Listing 8: pyproject.toml - Semantic Release Config

```
1 [tool.semantic_release]
2 version_variable = "src/__init__.py:__version__"
3 version_source = "tag"
4 commit_version_number = true
5 tag_commit = true
6 upload_to_pypi = false
7 upload_to_release = true
8
9 branch = "main"
10 hvcs = "github"
11
12 # Commit parsing
13 commit_parser = "angular"
14 major_on_zero = true
15 allow_zero_version = true
16
17 # Version bumping rules
18 [tool.semantic_release.commit_parser_options]
19 allowed_tags = [
20     "feat",      # New feature (minor bump)
21     "fix",       # Bug fix (patch bump)
22     "docs",      # Documentation only
23     "style",     # Code style changes
24     "refactor",  # Code refactoring
```

```

25     "perf",      # Performance improvements
26     "test",     # Test additions
27     "build",    # Build system changes
28     "ci",       # CI configuration
29 ]
30
31 major_tags = ["BREAKING CHANGE", "BREAKING"]
32 minor_tags = ["feat"]
33 patch_tags = ["fix", "perf"]

```

A.5 Abandonment Rationale - Detailed

This comprehensive GitHub Actions approach was abandoned due to several critical barriers:

1. Repository Modification Complexity

- Adding workflows requires write access to `.github/workflows/` directory
- Target repositories (LangChain, Intel XPU Backend) have existing CI/CD infrastructure
- Risk of conflicts with existing workflows, branch protection rules, and required checks
- Need to modify `.gitignore`, add `pyproject.toml`, and create metrics directories

2. Maintainer Approval Barriers

- External researchers cannot submit PRs that add observability workflows for academic purposes
- Maintainers would reject workflows that export metrics to external platforms (security/privacy)

- Request would require detailed justification, privacy impact assessment, and legal review
- No precedent for academic metric collection in production repositories

3. Per-Repository Customization Requirements

- Different projects use different languages (Python, C++, Rust), requiring tool variations
- Test frameworks vary: pytest, unittest, cargo test, gtest, etc.
- Build systems differ: setuptools, poetry, cargo, CMake, requiring custom metric extraction
- Each repository would need custom quality gate thresholds based on project maturity

4. Fork Limitations

- Forking allows workflow addition but creates isolation from production development
- Forks do not receive upstream commits in real-time, metrics become stale
- Fork metrics do not reflect actual project health, only snapshot quality
- Cannot access organizational secrets, runners, or internal deployment metrics

5. Computational and Financial Costs

- GitHub Actions minutes are limited for free accounts (2,000 minutes/month)
- Running comprehensive quality analysis on large codebases consumes significant minutes
- LangChain repository has 150+ contributors with dozens of daily commits

- Would require paid GitHub plan (\$4-\$21/user/month) for adequate compute quota

These constraints led to the pivot toward less invasive approaches in Phase 2 and Phase 3.

B Data Collection Scripts

The data collection process involved manually extracting data from GitHub’s web interface and organizing it into CSV format. While no automated scraping scripts were used due to permission constraints, the following describes the manual collection workflow:

B.1 GitHub Web UI Navigation

For each repository (LangChain and Intel XPU Backend), data was collected through:

1. Pull Request Data Collection

- Navigate to repository → Pull Requests tab
- Filter by date range (e.g., October 29 - November 27, 2025 for LangChain)
- Filter by state: `is:pr is:open`, `is:pr is:merged`, `is:pr is:closed`
- Copy PR titles, numbers, and timestamps to CSV

2. Issue Data Collection

- Navigate to repository → Issues tab
- Filter by creation date within analysis window
- Identify bug-related issues using keyword search: `bug`, `error`, `fix`, `broken`, `crash`, `fail`, `regression`, `exception`
- Export issue titles and creation timestamps

3. Repository Metadata

- Navigate to Insights → Pulse for activity summary
- Record commit hashes and dates from commit history
- Note relative timestamps ("2 days ago") and convert to absolute dates

B.2 CSV Data Organization

Data was organized into four CSV files per repository:

Listing 9: CSV File Structure

```
1 # opened_requests.csv
2 msg,hash,date_rel,date_abs
3 "Add_authentication_feature",#12345,"2 days ago","2025-11-27"
4
5 # merged_requests.csv
6 msg,hash,date_rel,date_abs
7 "Fix_authentication_bug",#12346,"1 day ago","2025-11-28"
8
9 # closed_requests.csv
10 msg,hash,date_rel,date_abs
11 "Update_documentation",#12347,"3 hours ago","2025-11-29"
12
13 # opened_issues.csv
14 msg,hash,date_rel,date_abs
15 "Bug: Login fails on Safari",#678,"1 week ago","2025-11-22"
```

C Analysis Source Code

The complete analysis pipeline is available in `examples/dora_metrics.py`. The script implements all DORA metric calculations and generates visualizations.

C.1 Key Functions

- `load_data(filepath)`: Loads CSV data and preprocesses timestamps
 - Parses `date_abs` column into datetime objects
 - Handles missing or malformed timestamps
 - Returns pandas DataFrame
- `calculate_deployment_frequency(merged_df, days)`: Calculates DF metric
 - Counts merged PRs per day over analysis window
 - Computes mean, median, and weekly aggregate
 - Classifies performance: Elite ($>1/\text{day}$), High (weekly), Medium (monthly), Low ($<\text{monthly}$)
- `calculate_lead_time(opened_df, merged_df)`: Calculates LT metric
 - Matches opened PRs with merged PRs using temporal proximity
 - Computes time delta in days
 - Returns median, mean, 25th/75th/90th percentiles
 - Classifies: Elite (<1 day), High (1-7 days), Medium (1-4 weeks), Low (>1 month)
- `calculate_change_failure_rate(issues_df, merged_df)`: Calculates CFR metric
 - Identifies bug issues using keyword matching on issue titles/descriptions
 - Counts bug issues within analysis window

- Computes ratio: $(\text{bug issues} / \text{total deployments}) \times 100\%$
- Classifies: Elite (0-15%), High (16-30%), Medium (31-45%), Low (>45%)
- `calculate_time_to_restore(issues_df, merged_df)`: Calculates MTTR metric
 - Attempts to pair bug issues with fix PRs
 - Searches for issue references in PR titles/bodies
 - Computes time from bug opened to fix merged
 - Returns median restoration time
 - Classifies: Elite (<1 hour), High (<1 day), Medium (1-7 days), Low (>1 week)
- `generate_visualizations(metrics_dict, output_dir)`: Creates charts
 - Deployment frequency time series
 - Lead time distribution histogram
 - Change failure rate comparison (deployments vs bugs)
 - Performance classification summary

C.2 Usage Example

Listing 10: Running DORA Metrics Analysis

```

1 import pandas as pd
2 from dora_metrics import *
3
4 # Load data
5 opened = load_data('data/langchain/opened_requests.csv')
6 merged = load_data('data/langchain/merged_requests.csv')
7 closed = load_data('data/langchain/closed_requests.csv')
8 issues = load_data('data/langchain/opened_issues.csv')

```

```

9
10 # Calculate metrics
11 df = calculate_deployment_frequency(merged, days=30)
12 lt = calculate_lead_time(opened, merged)
13 cfr = calculate_change_failure_rate(issues, merged)
14 mttr = calculate_time_to_restore(issues, merged)
15
16 # Generate visualizations
17 metrics = {'df': df, 'lt': lt, 'cfr': cfr, 'mttr': mttr}
18 generate_visualizations(metrics, 'output/langchain/')
19
20 # Print summary
21 print(f"Deployment_Frequency:_{df['mean']:.2f}/day")
22 print(f"Lead_Time_(median):_{lt['median']:.2f}_days")
23 print(f"Change_Failure_Rate:_{cfr['rate']:.2f}%")
24 print(f"Time_to_Restore:_{mttr['median']:.2f}_hours")

```

D Raw Data Samples

D.1 LangChain Repository Data

Sample entries from the LangChain analysis dataset (October 29 - November 27, 2025):

Table 5: Sample Merged Pull Requests (LangChain)

PR #	Title	Date Relative	Date Absolute
#25843	anthropic[patch]: Release 0.41.1	2 days ago	2025-11-25
#25841	docs: Update Anthropic tools example	2 days ago	2025-11-25
#25839	core[patch]: Fix streaming in RunnableBinding	3 days ago	2025-11-24
#25832	community[patch]: Add timeout to Redis	4 days ago	2025-11-23
#25829	docs: Add citation format to papers	5 days ago	2025-11-22

Table 6: Sample Bug Issues (LangChain)

Issue #	Title	Date Relative	Date Absolute
#4521	Bug: Streaming fails with Azure OpenAI	1 day ago	2025-11-26
#4518	Error when using callback with agents	3 days ago	2025-11-24
#4510	Fix: Memory leak in conversation buffer	5 days ago	2025-11-22
#4502	Regression: Tool calling broken in 0.41.0	1 week ago	2025-11-20

D.2 Intel XPU Backend Data

Sample entries from Intel XPU Backend analysis dataset (November 4 - December 2, 2025):

Table 7: Sample Merged Pull Requests (Intel XPU Backend)

PR #	Title	Date Relative	Date Absolute
#2104	Support for Triton 3.2.0	1 day ago	2025-12-01
#2101	Fix GEMM kernel for FP16	2 days ago	2025-11-30
#2098	Add support for grouped convolutions	4 days ago	2025-11-28
#2095	Optimize reduction kernels	6 days ago	2025-11-26
#2092	Fix memory allocation in compiler	1 week ago	2025-11-25

E OpenTelemetry Configuration Files

E.1 Docker Compose Configuration

The Phase 2 OpenTelemetry approach used the following Docker Compose setup:

Listing 11: docker-compose.yml for OTel Stack

```
1 version: '3.8'
2
3 services:
4   otel-collector:
5     image: otel/opentelemetry-collector-contrib:latest
6     container_name: otel-collector
7     command: ["--config=/etc/otel-collector-config.yaml"]
8     volumes:
9       - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
10    ports:
11      - "4317:4317"    # OTLP gRPC receiver
12      - "4318:4318"    # OTLP HTTP receiver
13      - "8888:8888"    # Prometheus metrics
14    networks:
15      - otel-network
16    depends_on:
17      - openobserve
18
19   openobserve:
20     image: public.ecr.aws/zinclabs/openobserve:latest
21     container_name: openobserve
22     ports:
23       - "5080:5080"
```

```

24     environment:
25         - ZO_ROOT_USER_EMAIL=admin@example.com
26         - ZO_ROOT_USER_PASSWORD=admin123
27         - ZO_DATA_DIR=/data
28     volumes:
29         - openobserve-data:/data
30     networks:
31         - otel-network
32
33 networks:
34     otel-network:
35         driver: bridge
36
37 volumes:
38     openobserve-data:

```

E.2 OpenTelemetry Collector Configuration

Listing 12: otel-collector-config.yaml

```

1 receivers:
2     github:
3         github_org: langchain-ai
4         search_query: "org:langchain-ai is:pr"
5         scrapers:
6             github:
7                 collection_interval: 60s
8         auth:
9             authenticator: bearertokenauth

```



```

10
11 extensions:
12     bearertokenauth:
13         token: "${GITHUB_TOKEN}"
14
15 processors:
16     batch:
17         timeout: 10s
18         send_batch_size: 100
19
20 exporters:
21     otlphttp:
22         endpoint: "http://openobserve:5080/api/default/"
23         headers:
24             Authorization: "Basic_${OPENOBSERVE_CREDENTIALS}"
25             stream-name: "github-metrics"
26
27 service:
28     extensions: [bearertokenauth]
29     pipelines:
30         metrics:
31             receivers: [github]
32             processors: [batch]
33             exporters: [otlphttp]

```

E.3 Performance Classification Logic

Listing 13: DORA Classification

```

1 def classify_deployment_frequency(deployments_per_day):
2     """Elite: >1/day, High: weekly, Medium: monthly, Low: <monthly
3         """
4     if deployments_per_day >= 1:
5         return 'Elite'
6     elif deployments_per_day >= 1/7:
7         return 'High'
8     elif deployments_per_day >= 1/30:
9         return 'Medium'
10    else:
11        return 'Low'
12
13 def classify_lead_time(median_days):
14     """Elite: <1 day, High: 1-7 days, Medium: 1-4 weeks, Low: >1
15         month"""
16     if median_days < 1:
17         return 'Elite'
18     elif median_days <= 7:
19         return 'High'
20     elif median_days <= 28:
21         return 'Medium'
22     else:
23         return 'Low'
24
25 def classify_change_failure_rate(percentage):
26     """Elite: 0-15%, High: 16-30%, Medium: 31-45%, Low: >45%"""
27     if percentage <= 15:
28         return 'Elite'
29     elif percentage <= 30:

```

```

28         return 'High'
29     elif percentage <= 45:
30         return 'Medium'
31     else:
32         return 'Low'
33
34 def classify_time_to_restore(median_hours):
35     """Elite: <1 hour, High: <1 day, Medium: 1-7 days, Low: >1 week
36         """
37     if median_hours < 1:
38         return 'Elite'
39     elif median_hours < 24:
40         return 'High'
41     elif median_hours < 168: # 7 days
42         return 'Medium'
43     else:
44         return 'Low'

```